1. Division Algorithm

```
        .orig   x3000
        NOT     R4, R1          ; let R4 = -R1; i.e., -divisor
        ADD     R4, R4, #1      ; via 2's complement
        ADD     R3, R0, #0      ; let R3 = R0; i.e., the remainder

LOOP    ADD     R2, R2, #1      ; counter+1 for each iteration, this is the quotient
        ADD     R3, R3, R4      ; subtract divisor from dividend
        BRn     DONE            ; if remainder becomes negative this iteration
                                ; jump to DONE to revert this iteration
        BRz     FIN             ; if remainder is 0, then finished
        BRp     LOOP            ; if remainder still positive, keep subtracting

DONE    ADD     R3, R3, R1      ; reverting
        ADD     R2, R2, #-1     ; and then finished

FIN     HALT
        .end
```

2. Euclidean Algorithm

```
        .orig x3000
BEG     ADD     R0, R2, #0
        NOT     R0, R0          ; let R0 = -R2
        ADD     R0, R0, #1

LOOP    ADD     R1, R1, R0      ; keep let R1 = R1 - R2
        BRp     LOOP            ; loop
        BRn     SWAP            ; until need to swap
        BRz     FIN             ; or until finished

SWAP    ADD     R1, R1, R2      ; add R2 back once so R1 is positive again
        ADD     R0, R2, #0      ; the swapping
        ADD     R2, R1, #0
        ADD     R1, R0, #0
        BRnzp   BEG             ; get back to beginning

FIN     HALT
        .end
```